

7.1.10 Hopfield Network *

Hopfield network is a special kind of neural network whose response is different from other neural networks. It is calculated by converging iterative process. It has just one layer of neurons relating to the size of the input and output, which must be the same. When such a network recognizes, for example, digits, we present a list of correctly rendered digits to the network. Subsequently, the network can transform a noise input to the relating perfect output.

In 1982, John Hopfield introduced an artificial neural network to collect and retrieve memory like the human brain. Here, a neuron is either on or off the situation. The state of a neuron (on

+1 or off 0) will be restored, relying on the input it receives from the other neuron. A Hopfield network is at first prepared to store various patterns or memories. Afterward, it is ready to recognize any of the learned patterns by uncovering partial or even some corrupted data about that pattern, i.e., it eventually settles down and restores the closest pattern. Thus, similar to the human brain, the Hopfield model has stability in pattern recognition.

A Hopfield network is a single-layered and recurrent network in which the neurons are entirely connected, i.e., each neuron is associated with other neurons. If there are two neurons i and j , then there is a connectivity weight w_{ij} lies between them which is symmetric $w_{ij} = w_{ji}$.

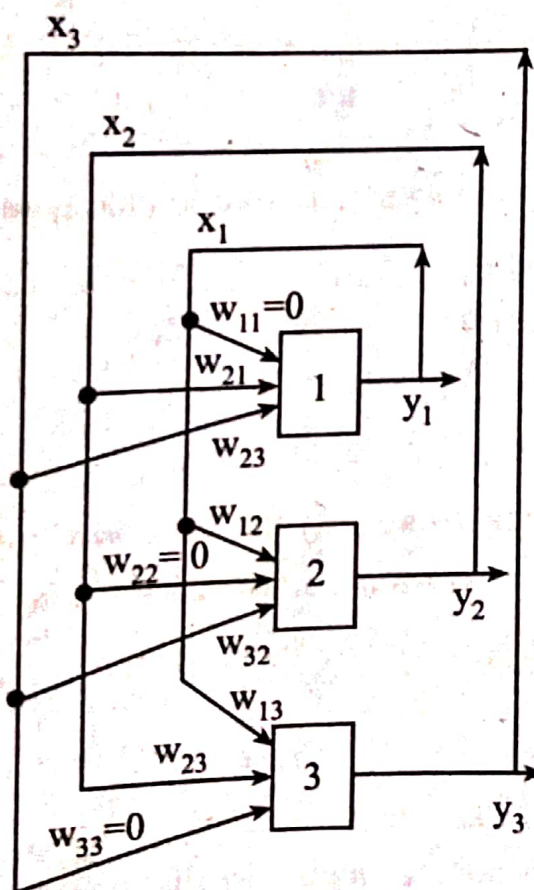


Figure 7.5: Hopfield network

Training algorithm:

For storing bipolar input patterns weight matrix is given by

$$W_{ij} = \sum_{P=1}^P s_i(P) s_j(P) \text{ for } i \neq j$$

$$W_{ij} = \sum_{P=1}^P s_i(P) s_j(P)$$

where $s_j(P)$ is input pattern, $s_i(P)$ is transverse of $s_j(P)$ & weights have no self-connection

Testing algorithm:

Step 0: Initialize weights to store pattern.

Step 1: When activation of network is not converged, performs step 2-6.

Step 2: Make initial activations of network equal to external input vectors x :

$$y_i = x_i \quad [i = 1 \text{ to } n]$$

$$y_i = x_i$$

Step 3: Perform steps 5-6 for each unit y . (units are updated in random order)

Step 4: Calculate net input of the network.

$$y_{in} = y_i + \sum_j y_j w_{ji}$$

$$y_{in} = y_i + \sum_j y_j w_{ji}$$

Apply activation over net input to calculate output.

$$y_i = \begin{cases} 1 & \text{if } y_{in} > \theta_i \\ y_i & \text{if } y_{in} = \theta_i \\ 0 & \text{if } y_{in} < \theta_i \end{cases}$$

θ_i is 0 basically.

Step 5: Now feedback (transmit) the obtained output to all other units. Thus, activation vectors are updated.

Step 6: Finally, test the network for convergence (are input vectors equal).